



Exercises session 5

Reductions, NP-hard, NP-complete

1 Definitions

Definition 1 (Computable functions) A function $f : \Sigma^* \rightarrow \Sigma^*$ is a **polynomial time computable function** if some polynomial time Turing machine exists that halts with just $f(w)$ on its tape, when started with the input $w \in \Sigma^*$.

Definition 2 (Karp reductions) Language A is **polynomial time Karp reducible** to language B , written $A \leq_P B$, if there is a polynomial time computable function $f : \Sigma^* \rightarrow \Sigma^*$ such that:

$$w \in A \iff f(w) \in B.$$

Definition 3 (NP) A language L is in NP if there exists a polynomial time verifier V such that for every $w \in \Sigma^*$:

$$w \in L \iff \exists c \text{ with } |c| = \text{poly}(|w|) \text{ and } V(w, c) = 1.$$

The string c is called a *certificate* (or *witness*) for w .

Definition 4 (P) A language L is in P if there exists a deterministic Turing machine that decides L in polynomial time.

Definition 5 (NP-Hardness) A language B is **NP-hard** if for any $A \in NP$ we have $A \leq_P B$.

Definition 6 (NP-Completeness) A language B is **NP-complete** if B is NP-hard and $B \in NP$.

Theorem 1 (Cook-Levin) SAT is NP-complete.

2 Session exercises

1. Show that if L is NP-hard and $L \in P$, then $P = NP$.

Hint: use the definition of NP-hardness together with the fact that polynomial-time reductions compose with polynomial-time algorithms.

2. Prove that if $P = NP$, then every language in P , except $\emptyset$ and Σ^* , is NP-complete.

Hint: you need to show both NP-hardness and membership in NP. For the hardness direction, think carefully about why $\emptyset$ and Σ^ must be excluded.*

3. Let A, B, C be languages such that $A \leq_P B$ and $B \leq_P C$. Show that $A \leq_P C$.

Write the composed reduction function explicitly and verify that it runs in polynomial time.

4. Show that if $A \leq_P B$ and A is NP-hard, then B is NP-hard.

Follow-up: what additional condition do you need on B to conclude that B is NP-complete?

5. For each statement below, prove whether it is true or false.

(a) For every NP-hard problem X , we have $X \leq_P \text{SAT}$.

(b) Let X be an NP-complete problem. If there exists an algorithm to solve X in polynomial time, then SAT can be solved in polynomial time.

6. Let M be a Turing machine and t a number given in unary. Consider the following problem:

$$\text{ACCEPTS@}t = \{\langle M, t \rangle : M \text{ accepts some input in less than } t \text{ steps}\}$$

Prove by definition that $\text{ACCEPTS@}t$ is NP-complete.

Recall that to prove NP-completeness you need to show (i) membership in NP — what is the certificate? — and (ii) NP-hardness — every NP language reduces to it.